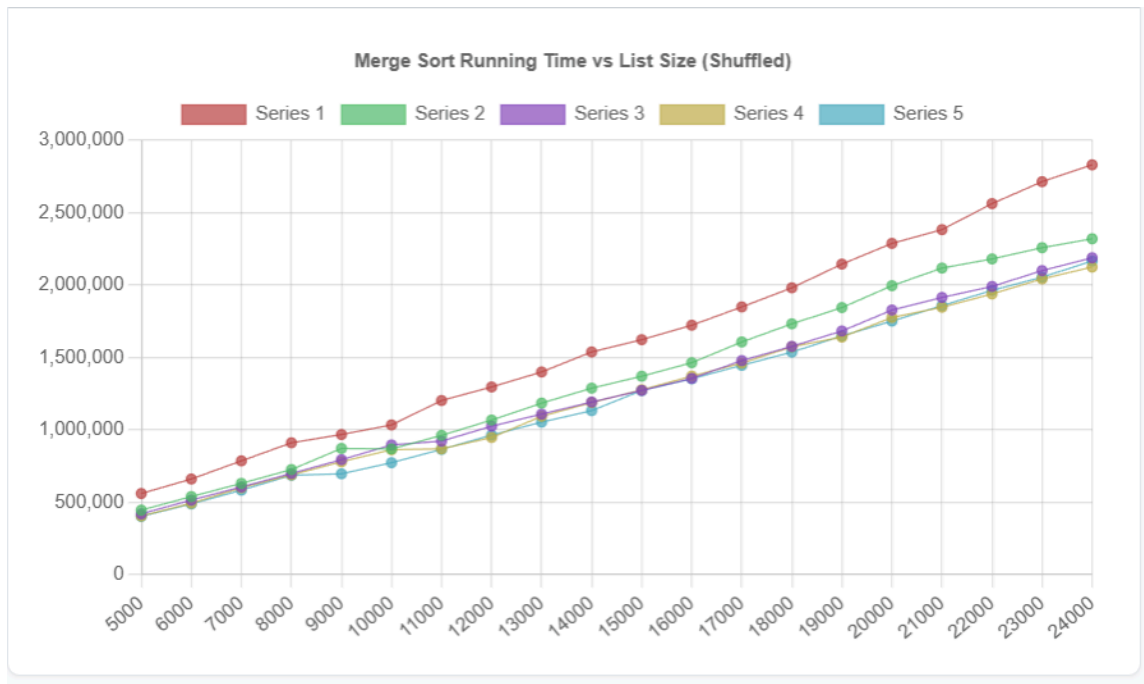


Assignment 5: Sorter

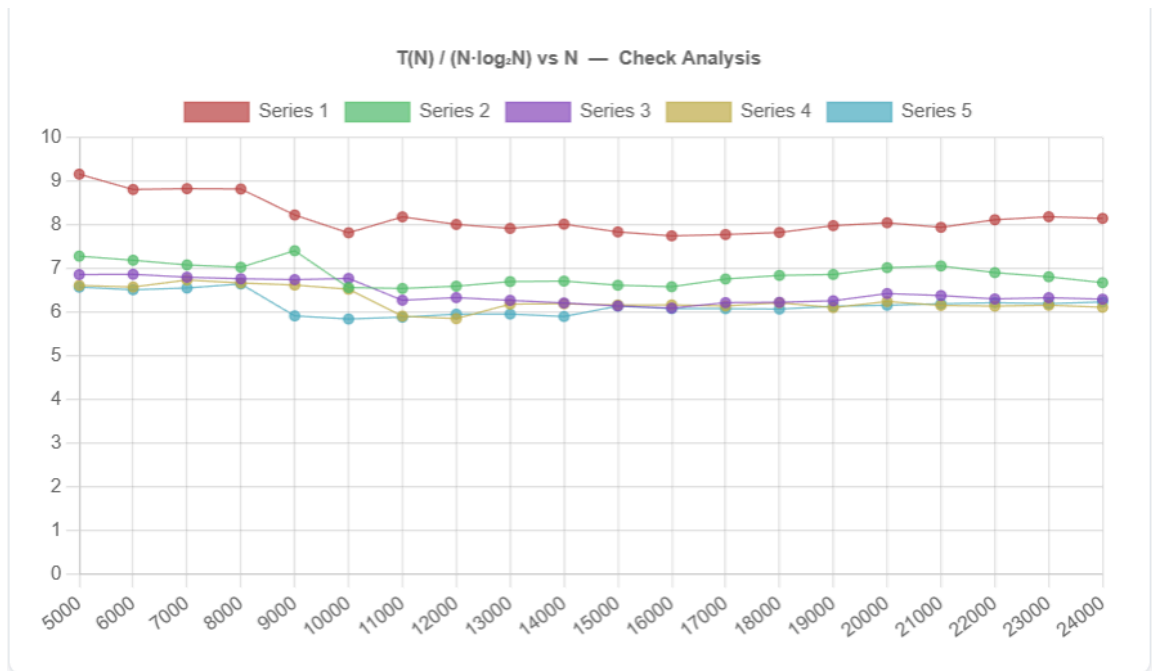
1. My programming partner is Zewei Ren and I submitted the program to Gradescope.
2.
 - 2.1. We applied pair programming techniques effectively in this assignment; I used these techniques to write about 65% of the code.
 - 2.2. You learned from the teaching assistant and the students in Lab 4 that we can learn from our programming partners, adopting their good programming approaches and habits. In this assignment, I learned from my programming partner the importance of maintaining clear and rigorous programming logic.
 - 2.3. It took me about 5 hours to complete the programming and testing parts of this assignment.
 - 2.4. I felt that my programming partner and I communicated very frequently during the programming process, and I think he had a very positive attitude towards the assignment. I will continue to collaborate with him.
3. The expected growth rate of merge sort is $O(N \log N)$ in the best, worst, and average cases. The ordering of the list has little effect on the running time of merge sort. Unlike quicksort, merge sort always divides the list into two equal halves and merges them regardless of the input order, so its growth rate remains $O(N \log N)$ in the best, average, and worst cases.
4. To collect the runtime of merge sort on three different ordered lists, we implemented three experimental classes: *MergeSortShuffledTimingExperiment* , *MergeSortDescendingTimingExperiment* and *MergeSortNearlyAscendingTimingExperiment*. All are inherited from the course provided *TimingExperimentFramework*. The parameters for all three experiments are identical: list size ranges from 5,000 to 24,000, increasing by 1,000 each time, for a total of 20 problem sizes; each problem size is run 50 times and the median is taken, in nanoseconds, to reduce the impact of JVM warm-up and system noise. The only difference between the three experiments is...*setupExperiment* The method for generating lists: Randomized experiments are conducted using...*generatePermutedArrayListGenerate* a randomly shuffled list; used for descending order experiments.*generateDescendingArrayListGenerate* a list in complete reverse order; used for approximately ascending order experiments.*generateNearlyAscendingArrayListGenerates* an approximately ordered list with a small number of inversions. A new list of the corresponding

type is generated in each iteration to ensure that the unsorted original data is used for each timing step. (Timing section)runComputationContains only onceMergeSorter.sortThe call. All three experiments used threshold construction.MergeSorterWhen the size of the sublist falls below a threshold, switch to insertion sort to achieve optimal performance. In the two graphs below, the line charts for Series 1-5 correspond to...threshold By analyzing the running times of 1, 16, 19, 24, and 28 on a randomized list, we can see that the fastest running time is when the threshold is 24.



In the graph below, we observe that the ratio of runtime to our expected growth rate $O(N \log N)$ tends to be constant as the problem size increases. Therefore, the plotted runtime growth rate aligns with my predicted Big O

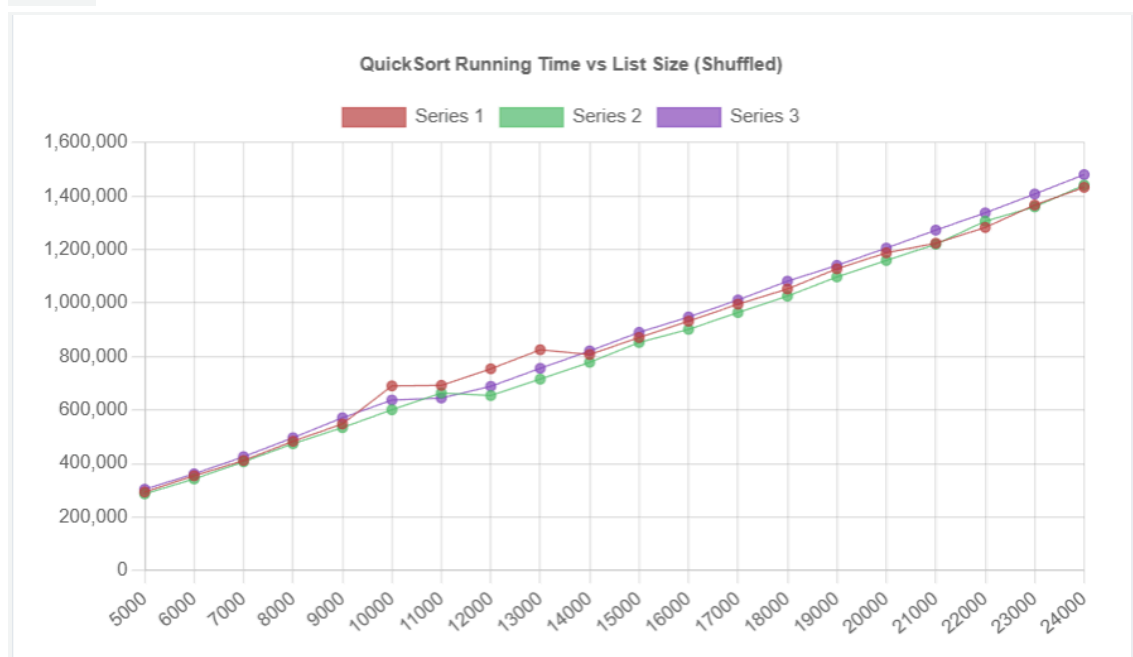
behavior.



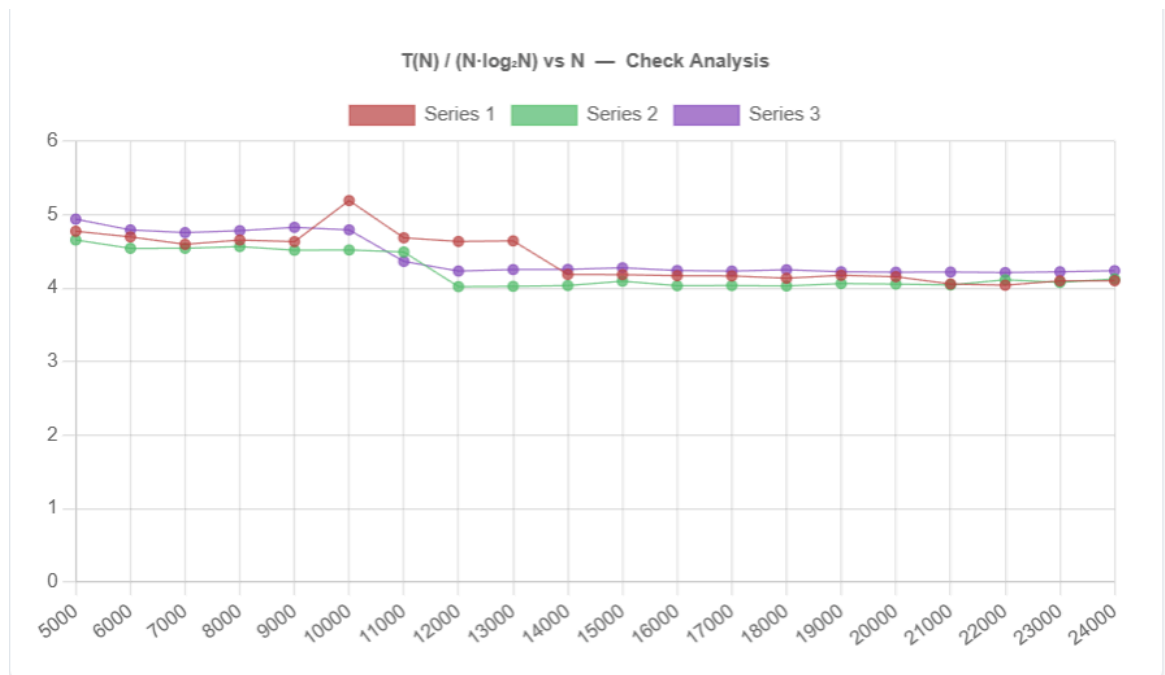
5. Quicksort has an expected growth rate of $O(N \log N)$ in the best and average cases, and $O(N^2)$ in the worst case. The list order affects the running time by influencing the choice of the pivot, i.e., whether the number line can divide the list into approximately equal halves. Regarding pivot selection:
 - FirstPivotChooser (Select the first element) It performs reasonably well for random lists, but causes worst-case $O(N^2)$ results for sorted, nearly ordered, or reversed lists because the partitioning is extremely uneven each time.
 - RandomPivotChooser (random selection) Regardless of the input order, it is expected to partition the data evenly, with an average stability of $O(N \log N)$, but it introduces the additional overhead of random number generation.
 - MedianOfThreePivotChooser (Three-number middle picker) Taking the median of the first, middle, and last elements as the pivot, this method works best for ordered and nearly ordered lists. In practice, it is usually the fastest of the three strategies and has no random number overhead.

6. PivotChooserSubclass instances are called strategy object. Because it implements the Strategy Pattern—encapsulating a part of the algorithm (pivot selection) as a replaceable object, through...QuickSortConstructor injection. The following code...The `QuickSorter<String> qs = new QuickSorter<String>((list, leftIndex, rightIndex) -> rightIndex);` method can call the quicksort constructor because...PivotChooserIt isFunctional interfaceMoreover, it only has one abstract method.getPivotIndexJava allows the use of lambda expressionInstead of a concrete class instance that implements the interface.

7. To collect the running time of quicksort on three different ordered lists, we implemented three experimental classes: `QuickSortShuffledTimingExperiment`, `QuickSortDescendingTimingExperiment` and `QuickSortNearlyAscendingTimingExperiment`. All are inherited from the course provided `TimingExperimentFramework`. The parameters for all three experiments are identical: list size ranges from 5,000 to 24,000, increasing by 1,000 each time, for a total of 20 problem sizes; each problem size is run 50 times and the median is taken, in nanoseconds, to reduce the impact of JVM warm-up and system noise. The difference between the three experiments lies in...
`setupExperiment` The method for generating lists: `Randomized` experiments are conducted using...
`generatePermutedArrayList` Generate a randomly shuffled list; used for descending order experiments.
`generateDescendingArrayList` Generate a list in complete reverse order; used for approximately ascending order experiments.
`generateNearlyAscendingArrayList` Generates an approximately ordered list with a small number of inversions. A new list of the corresponding type is generated in each iteration to ensure that the unsorted original data is used for each timing step. (Timing section)
`runComputation` Contains only once `QuickSorter.sort` The call to [the system/mechanism] was performed. To compare the performance of the three pivot strategies, each experiment modified the static variable [the system/mechanism].`pivotChooser` Use respectively `MedianOfThreePivotChooser`, `RandomPivotChooser` and `FirstPivotChooser`. The algorithm was run once in each of the three list orders, collecting a total of nine sets of data. In the two graphs below, the line segments Series 1-3 correspond to the runtime of `MedianOfThree`, `FirstPivot`, and `RandomPivot` on the randomized list. We can observe that the runtime of these three pivot selection methods on the randomized list is roughly the same.

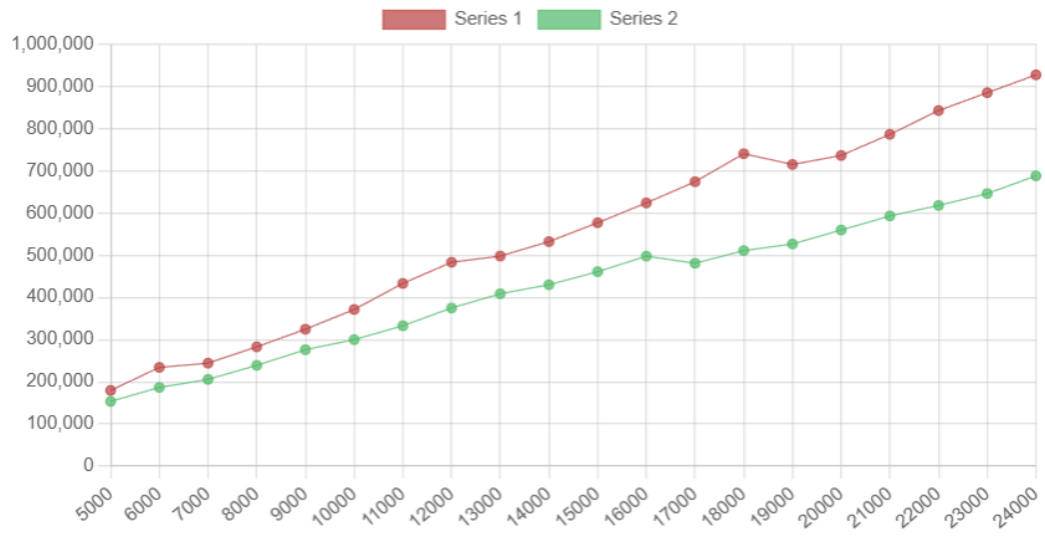


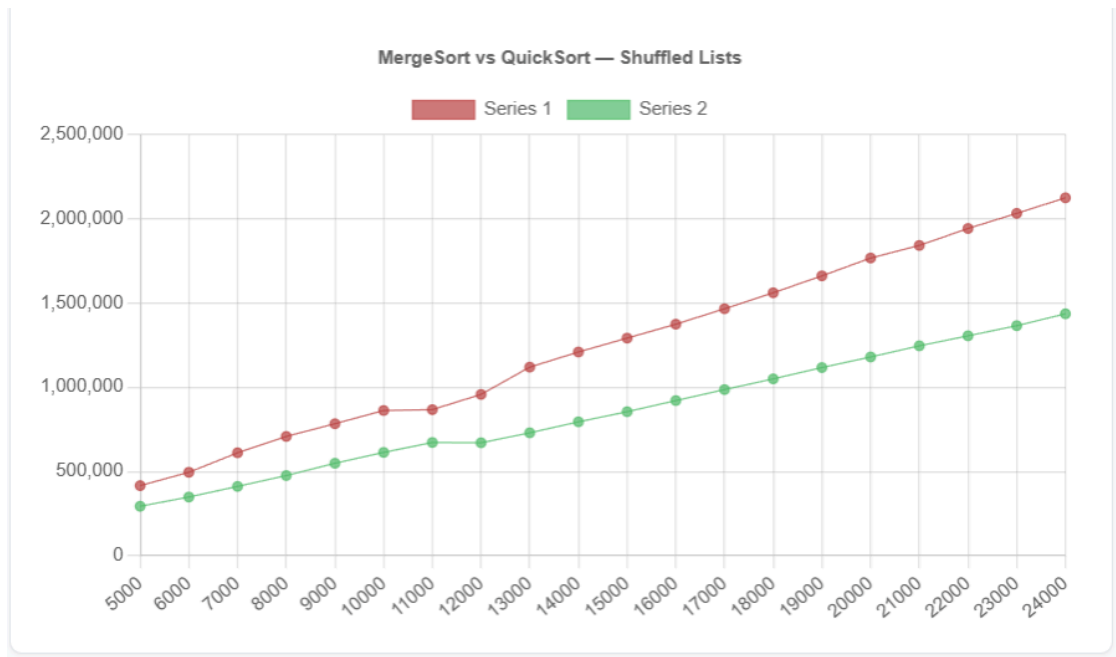
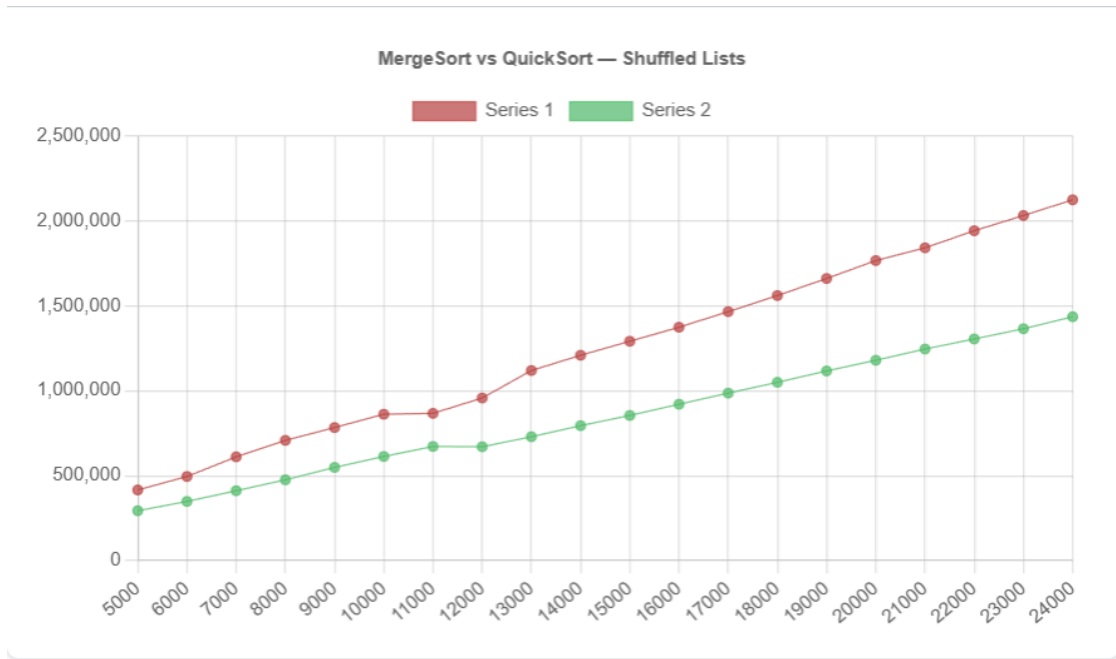
In the graph below, we observe that the ratio of runtime to our expected growth rate $O(N \log N)$ tends to be constant as the problem size increases. Therefore, the plotted runtime growth rate aligns with my predicted Big O behavior.



8. To compare the performance of merge sort and quicksort on three different ordered lists, we implemented six experimental classes, each corresponding to one of the two sorting algorithms and one of the three list types, all inheriting from the classes provided in the course. `TimingExperimentFramework`. The parameters for all six experiments were identical: list sizes ranged from 5,000 to 24,000, increasing by 1,000 each time, for a total of 20 problem sizes; each problem size was run 50 times and the median was taken, in nanoseconds, to reduce the impact of JVM warm-up and system noise. The three list types were distinguished by different generation methods: randomized lists
`used...generatePermutedArrayListDescending` sequence list
`usagegenerateDescendingArrayListApproximate` ascending sequence list
`usesgenerateNearlyAscendingArrayList` Each iteration is `insetupExperimentA` new list of the corresponding type is regenerated to ensure that the unsorted original data is used for each timed operation. The merge sort experiment uses a threshold of 24 for construction. `MergeSorter` The quicksort experiment selects the optimal pivot strategy based on the list type: disordered and approximately ascending lists use `...MedianOfThreePivotChooserDescending` sequence lists use `RandomPivotChooser`. Based on the experimental results, `RandomPivotChooser` outperforms `MedianOfThreePivotChooser` on descending lists. `Timing` section `runComputationContains` only once `sort` The call. In the three graphs below, Series 1 and 2 correspond to the runtime of Merge Sort and Quick Sort, respectively.

MergeSort vs QuickSort — Descending Lists





9. In the three graphs below, Series 1 and 2 correspond to the runtime of Merge Sort and Quick Sort, respectively. We find that as the problem size increases, the ratio of the runtime to our expected growth rate $O(N \log N)$ tends to be constant. Therefore, the plotted runtime growth rate is consistent with my predicted Big O behavior. Both merge sort and quicksort are comparison-based divide-and-conquer algorithms whose theoretical complexities are derived from their recursive structure. Since our implementation correctly follows these structures, the actual running times match the expected $O(N \log N)$ growth rate. The slight variations at smaller problem sizes are due to JVM warm-up and system noise, which diminish as the problem size increases.

